

CSE445 Lecture 5

Loss Functions and Maximum Likelihood

M.A.K, ECE, NSU

Why Do We Need a Loss Function?

A machine learning model makes a prediction, but during training we need to know:

How wrong is the model?

A **loss function** gives a numerical answer to this question.

Abstract Prediction Error $\Rightarrow$ A Numeric Quantity $\Rightarrow$ Optimization Guidance

Smaller loss means the model predictions are closer to the correct answers. **Training is almost complete.**

Training Data and Model Notation

Assume we have I training examples:

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^I$$

Where $\mathbf{x}_i$ is the input feature and y_i is the true output.

The model predicts:

$$\hat{y}_i = f(\mathbf{x}_i; \boldsymbol{\theta})$$

Here, $\boldsymbol{\theta}$ represents the parameters learned by the model.

What Does a Loss Function Do?

For one example, the loss compares the **true output** and the **predicted output**:

$$\ell(y_i, \hat{y}_i)$$

For the full training dataset, we usually average the losses:

$$L(\boldsymbol{\theta}) = \frac{1}{I} \sum_{i=1}^I \ell(y_i, f(\mathbf{x}_i; \boldsymbol{\theta}))$$

$L(\boldsymbol{\theta})$ is a single scalar number.

A good model should have a **small loss** on the training data and also on new data.

Training Means Minimizing Loss

Training means finding model parameters that make the loss small.

$$\theta^* = \arg \min_{\theta} L(\theta)$$

In simple words:

Try Parameters → Make Predictions → Compute Loss → Update Parameters

This is the basic idea behind learning or fitting a model.

Types of Loss Values

Different machine learning tasks need different loss functions.

Task	Output type	Example
Regression	Real number	House price
Binary classification	Two classes	Spam / Not Spam
Multi-class classification	Many classes	Music Genre

The loss function should match the type of output the model predicts.

Regression: Predicting a Real Number

In regression, the output is a real number.

$$y_i \in \mathbb{R}, \quad \hat{y}_i = f(\mathbf{x}_i; \boldsymbol{\theta})$$

A common loss is the **squared error**:

$$\ell(y_i, \hat{y}_i) = (y_i - \hat{y}_i)^2$$

The full training loss is:

$$L(\boldsymbol{\theta}) = \frac{1}{I} \sum_{i=1}^I (y_i - f(\mathbf{x}_i; \boldsymbol{\theta}))^2$$

Why Squared Error?

Squared error is useful because it strongly penalizes large mistakes.

True value	Prediction	Squared error
10	9	$(10 - 9)^2 = 1$
10	7	$(10 - 7)^2 = 9$
10	2	$(10 - 2)^2 = 64$

A larger error becomes much larger after squaring.

So least squares encourages the model to avoid **large prediction errors**.

Log and Exponential Functions

Maximum likelihood often uses `log` and `exp` functions.

They are inverse operations:

$$\log(\exp(a)) = a, \quad \exp(\log(a)) = a \quad (a > 0)$$

Two useful rules:

$$\log(ab) = \log(a) + \log(b)$$

$$\log(a^b) = b \log(a)$$

These rules help us turn products into sums, which are easier to compute and optimize.

From Probability to Prediction

A model can predict a number directly, but many loss functions are easier to design from probability.

Instead of only predicting one value, we can ask the model to predict:

$$p(y \mid \mathbf{x}; \boldsymbol{\theta})$$

This means:

The probability of output y given input $\mathbf{x}$ and parameters $\boldsymbol{\theta}$

Training then tries to make the observed correct outputs have **high probability**.

Probability of the Training Data: Likelihood

Likelihood of the training data is the ground truth. For one training example, the model assigns probability:

$$p(y_i \mid \mathbf{x}_i; \boldsymbol{\theta})$$

For the full dataset, it is called the likelihood - the probability of all the samples (dataset):

$$\mathcal{L}(\boldsymbol{\theta}) = \prod_{i=1}^I p(y_i \mid \mathbf{x}_i; \boldsymbol{\theta})$$

Therefore, $\mathcal{L}(\boldsymbol{\theta})$ estimates the probability of the entire dataset.

Maximum likelihood estimation finds parameters that make the observed data most likely:

$$\boldsymbol{\theta}^* = \arg \max_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta})$$

Likelihood to Log Likelihood

The likelihood is a product of many probabilities. Products are hard to handle. Also, the probability values are smaller than 1. Therefore, likelihood values decreases if the samples in the dataset increases.

A better approach is to use sum. If the samples increases, summation value also increases. Also, sums are easier to manage for mathematical optimization. That is why log function is used to convert likelihood from products of probabilities to sums of probabilities:

$$\log \mathcal{L}(\boldsymbol{\theta}) = \sum_{i=1}^I \log p(y_i \mid \mathbf{x}_i; \boldsymbol{\theta})$$

Because $\log(\cdot)$ is monotonically increasing, maximizing the likelihood is equivalent to maximizing the log-likelihood.

Since probabilities are usually less than 1, their logarithms are negative. Therefore, the log-likelihood is usually negative.

So, instead of maximizing log-likelihood, we often minimize the **negative log-likelihood**.

Using Negative Log Likelihood as Loss

Machine learning usually formulates training as **minimizing a loss**, not maximizing a score.

Maximum likelihood maximizes the log-likelihood. Since log-likelihood values are often negative, we convert the problem into minimization by taking its negative.

Thus, we minimize the **negative log-likelihood**:

$$L(\boldsymbol{\theta}) = - \sum_{i=1}^I \log p(y_i \mid \mathbf{x}_i; \boldsymbol{\theta})$$

High probability for correct outputs $\Rightarrow$ **Small negative log likelihood**

This connects probability modeling with ordinary loss minimization.

Recipe for Building a Loss Function

A simple recipe:

- ① Identify the output type: **real number**, **two classes**, or **many classes**.
- ② Choose a probability model suitable for that output.
- ③ Use the machine learning model to estimate the probability model's parameters from the input features.
- ④ Minimize the **negative log-likelihood**.

The goal is always the same:

Make the observed training outputs as probable as possible.

MLE for Regression: Designing the Loss

Regression predicts a **real-valued output**. To design a regression loss using MLE, we first describe how the target value is generated around the model prediction.

A common assumption is:

$$y_i = f(\mathbf{x}_i; \boldsymbol{\theta}) + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma^2)$$

Equivalently,

$$y_i \mid \mathbf{x}_i, \boldsymbol{\theta} \sim \mathcal{N}(f(\mathbf{x}_i; \boldsymbol{\theta}), \sigma^2)$$

This means:

The model prediction is the mean of a Gaussian distribution,
and the observed target is expected to be close to that mean.

So, instead of choosing a loss randomly, we choose a probability model for the output and derive the loss from **maximum likelihood**.

Regression Loss from MLE

For independent training examples, the likelihood is

$$p(\mathbf{y} \mid \mathbf{X}, \boldsymbol{\theta}) = \prod_{i=1}^I p(y_i \mid \mathbf{x}_i, \boldsymbol{\theta})$$

Using the Gaussian assumption, the negative log-likelihood becomes

$$-\log p(\mathbf{y} \mid \mathbf{X}, \boldsymbol{\theta}) = \frac{1}{2\sigma^2} \sum_{i=1}^I (y_i - f(\mathbf{x}_i; \boldsymbol{\theta}))^2 + \text{constant}$$

The constant and the factor $\frac{1}{2\sigma^2}$ do not change the best parameters. Therefore, minimizing the negative log-likelihood is equivalent to minimizing

$$L(\boldsymbol{\theta}) = \frac{1}{I} \sum_{i=1}^I (y_i - f(\mathbf{x}_i; \boldsymbol{\theta}))^2$$

Binary Classification

In binary classification, the output has two possible classes:

$$y_i \in \{0, 1\}$$

The model predicts a probability:

$$\hat{p}_i = P(y_i = 1 \mid \mathbf{x}_i; \boldsymbol{\theta})$$

Because a probability must be between 0 and 1, we often use the sigmoid function:

$$\hat{p}_i = \sigma(z_i) = \frac{1}{1 + \exp(-z_i)}$$

Binary Cross-Entropy Loss

For binary classification, the negative log likelihood becomes:

$$L(\boldsymbol{\theta}) = -\frac{1}{I} \sum_{i=1}^I \left[y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i) \right]$$

This is called **binary cross-entropy loss**.

If the true label is $y_i = 1$, the loss wants $\hat{p}_i$ close to 1.

If the true label is $y_i = 0$, the loss wants $\hat{p}_i$ close to 0.

Binary Prediction Rule

After training, the model gives a probability $\hat{p}_i$.

A simple decision rule is:

$$\hat{y}_i = \begin{cases} 1, & \hat{p}_i \geq 0.5 \\ 0, & \hat{p}_i < 0.5 \end{cases}$$

Example:

Predicted probability	Predicted class
$\hat{p} = 0.82$	1
$\hat{p} = 0.31$	0

Multi-class Classification

In multi-class classification, there are K possible classes:

$$y_i \in \{1, 2, \dots, K\}$$

The model predicts one probability for each class:

$$\hat{\mathbf{p}}_i = [\hat{p}_{i1} \quad \hat{p}_{i2} \quad \cdots \quad \hat{p}_{iK}]$$

These probabilities must satisfy:

$$0 \leq \hat{p}_{ik} \leq 1, \quad \sum_{k=1}^K \hat{p}_{ik} = 1$$

Softmax Converts Scores to Probabilities

A neural network may output any real-valued scores:

$$z_{i1}, z_{i2}, \dots, z_{iK}$$

The **softmax function** converts these scores into probabilities:

$$\hat{p}_{ik} = \frac{\exp(z_{ik})}{\sum_{j=1}^K \exp(z_{ij})}$$

Softmax makes every probability positive and makes all probabilities sum to 1.

The largest score usually becomes the largest probability.

Multi-class Cross-Entropy Loss

If the true class for example i is y_i , then the loss is:

$$\ell_i = -\log(\hat{p}_{i,y_i})$$

For the full dataset:

$$L(\boldsymbol{\theta}) = -\frac{1}{I} \sum_{i=1}^I \log(\hat{p}_{i,y_i})$$

This is called **multiclass cross-entropy loss**.

It becomes small when the model gives high probability to the correct class.

Multi-class Prediction Rule

After training, choose the class with the largest predicted probability:

$$\hat{y}_i = \arg \max_k \hat{p}_{ik}$$

Example for three music genres:

Class	Predicted probability
Rock	0.15
Classical	0.70
Jazz	0.15

The prediction is **Classical** because it has the largest probability.

Multiple Outputs: Basic Idea

Sometimes a model predicts more than one output.

Example: predict both **height** and **weight**.

A simple approach is to add the losses for each output:

$$L(\boldsymbol{\theta}) = L_1(\boldsymbol{\theta}) + L_2(\boldsymbol{\theta}) + \cdots + L_D(\boldsymbol{\theta})$$

Important practical point:

If outputs have very different scales, standardize or rescale them before training.

How Loss Helps Machine Learning

The loss function gives the model a direction for improvement.

During training, algorithms such as gradient descent repeatedly update parameters:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} L(\boldsymbol{\theta})$$

where η is the learning rate.

loss $\Rightarrow$ gradient $\Rightarrow$ parameter update

Without a loss function, the model would not know how to improve.

Choosing the Right Loss Function

Choose the loss based on the output type.

Problem	Model output	Common loss
Regression	real value	squared error
Binary classification	probability of class 1	binary cross-entropy
Multiclass classification	class probabilities	multiclass cross-entropy

The main idea is simple:

a good loss function should reward correct predictions and penalize wrong predictions.

Summary

- A **loss function** measures how bad a model is.
- Training means finding θ that minimizes the loss.
- **Maximum likelihood** chooses parameters that make the training data most probable.
- **Negative log likelihood** turns maximum likelihood into a minimization problem.
- Squared error, binary cross-entropy, and multiclass cross-entropy are common losses.

Loss functions connect prediction, probability, and learning.